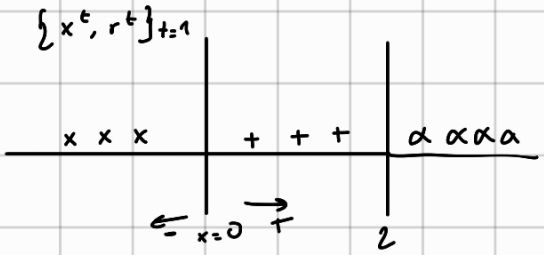
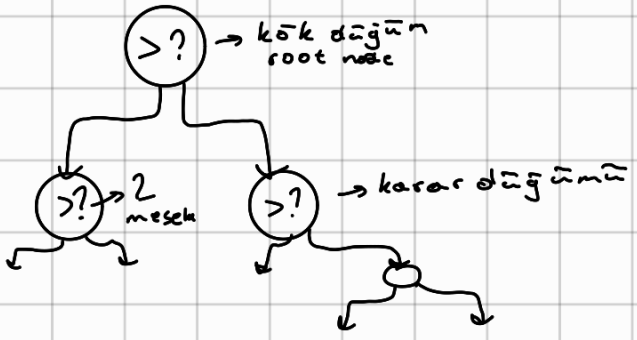
Ders6

Karar ağaçları (Decision tree)

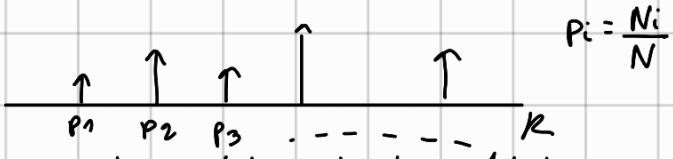




Amaç en sonda verileri homojen olarak ayırmak

Saflık ölçütleri (Impurity measures):

$p_i = \frac{N_i}{N}$



en saf durum tek bir olasılığın 1 kalanın

Entropy: $I_{entropy} = -\sum_{i=1}^{k} p_i \log p_i$

Gini Sabiti: $I_{Gini} = 1 - \sum_{i=1}^{k} p_i^2$

0 ile $1 - \frac{1}{N^2}$ arasında değer alır

Tek bir sınıf varsa entropy min

Çok sınıf varsa max olur

dallanırken max entropy nerdeyse

ordan devamet

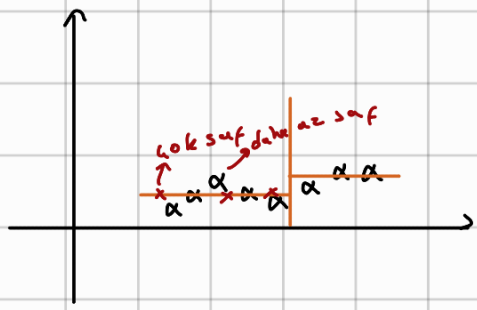
$\sum_{j=1}^{N} w_j I_j \quad w_j = \frac{N_j}{N}$

$\varepsilon = \varepsilon_k - \varepsilon_{k-1}$

while $\varepsilon > \gamma$ threhold dallan

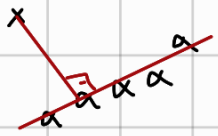
$L(\theta|X) = \sum_{j=1}^{N} w_j I_j + \underbrace{M}_{\text{overfittingi engeller}} \overbrace{\text{yaprak sayısı} \cdot \lambda \text{ sabit sayısı}}^{\lambda \cdot f(x)}$

Regresyon ağaçları



Metrik olarak varyans kaybilire, vega L1 kullanılabilir. (Goodness of split)

L1 norm aykırı değerlere hassasiyeti az



Avantajları

- Basittir. Görsel temsili kolaydır
- Açıklanması en kolay makine öğrenmesi
- Hem sınıflandırma hem regresyon için kullanılabilir
- Doğrusal olmayan ilişkileri bulabilir

Dezavantajları

Overfitting: Çok sayıda düğümden oluşuyor ise overfitting olabilir

Kararsızlık: Verilerdeki ufak değişiklik tamamen farklı bir ağaç oluşmasına neden olabilir.

Çözüm:

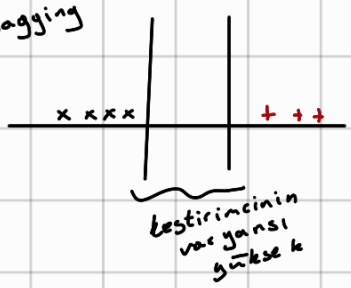
1- overfitting için budama yapılabilir

Ön budama: önceden max derinlik vererek ağacın uzaması engellenir

Son budama: Tam gelişmiş ağaçtan önemli katkı sağlamayan dalları kaldırır

Ensemble Learning

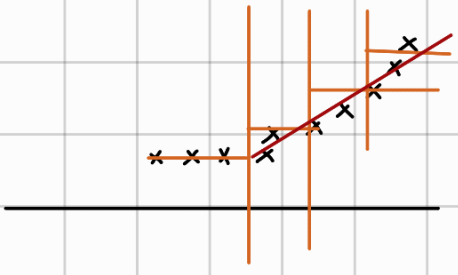
Bagging



100 tane karar ağacı alırım, yeni bir örnek geldiği zaman oylama yapıyorum böylece kestirimcinin varyansı küçülür.

Baggingle olan eğitim seti alt eğitim setlerine bölünür. Her veri seti ile Model eğitilir, böylece alt veri setleri kadar model oluşturulur. Tahminlerin birleştirilmesi aşmasında regresyon ağaçları için ortalama alınırken sınıflandırma ağaçlarında sonuçlar oylama ile belirlenir.

Boosting: Verinin özel bir bölgesine odaklanarak daha iyi bir sınıflandırıcı varsa bu şekilde yaklaşabiliriz



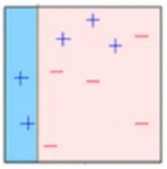
w_0 w_0 $w_0 + w_1 x$

2. Yükseltme(Boosting)

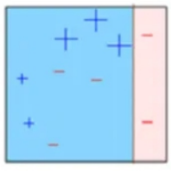
Yükseltme yönteminde temel fikir, veri setine farklı ağırlıklar verilmesi sonucu elde edilen ağaçlar topluluğundan çıkarsamalar yapılmasıdır. Başlangıçta tüm gözlemler eşit ağırlığa sahiptir. Ağaç topluluğu büyümeye başladıkça, problem bilgisine kurulu olarak ağırlıklandırmalar düzenlenir. Yanlış sınıflandırılan gözlemlerin ağırlığı arttırılırken, nadiren yanlış sınıflandırılan gözlemlerin ağırlığı azaltılır. Bu sayede ağaçlar zor durumlar karşısında kendini düzenleyebilme yeteneği kazanır.

Yükseltme yönteminin nasıl çalıştığını aşağıdaki adımlarla anlayalım.

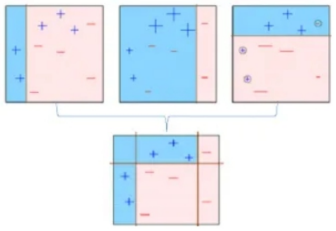
- Orijinal veri kümesinden bir alt küme oluşturulur.
- Başlangıçta, tüm veri noktalarına eşit ağırlık verilir.
- Bu alt kümede bir temel model oluşturulur.
- Bu model, tüm veri kümesi için tahminler yapmak için kullanılır.

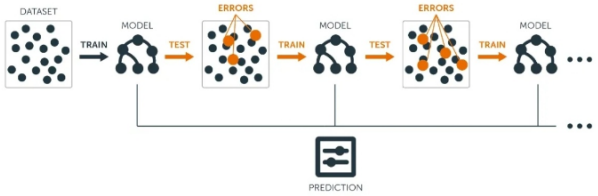


- Hatalar gerçek değerler ve öngörülen değerler kullanılarak hesaplanır.
- Yanlış öngörülen gözlemlere daha fazla ağırlık verilir. (Burada, sınıflandırılmamış olan üç tane mavi artı noktaya daha fazla ağırlık verilecektir.)



- Başka bir model oluşturulur ve veri setinde tahminler yapılır. (Bu model önceki modeldeki hataları düzeltmeye çalışır.)
- Benzer şekilde, her biri önceki modelin hatalarını düzelten çoklu modeller oluşturulur.
- Son model (güçlü öğrenen), tüm modellerin ağırlıklı ortalamasıdır.





kdnuggets.com'dan alınmıştır.

Adaboost

$f_i(x), i \in \{0, \dots, M\}$ Aynı modelin parametrelerini değiştirip lineer kombinasyonlarını kullanıyoruz

$$y = \sum_{i=1}^{m} w_i f_i(x)$$

verinin hangi bölgesinde hangi fonksiyon daha iyi

$$\mathcal{L}(\theta|X) = \sum_{i=1}^{N} w_t (\hat{r}^t - r^t)^2$$

↑ risk gibi

Bu modelde eğitim kümesi önce zayıf öğrenici ile eğitilir. Eğitim sonrası yanlış olarak tahminlenen örnekler bu algoritma için önemlidir. Bir sonraki eğitimde ilk tahminlemede yanlış öğrenilen eğitim verilerine daha fazla öncelik verilerek yani ağırlıkları artırarak tekrar eğitilir.

Ders 7

Boyut Azaltma

(1) Feature selection

$$\{x^t, r^t\}_{t=1}^{N}$$

$x^t_{N1\times 1} \rightarrow x^t_{N_2 \times 1}$ $\quad N_2 < N_1$

(2) Subset Selection

$F = \emptyset$

while not converge:

for $x_i \notin F$

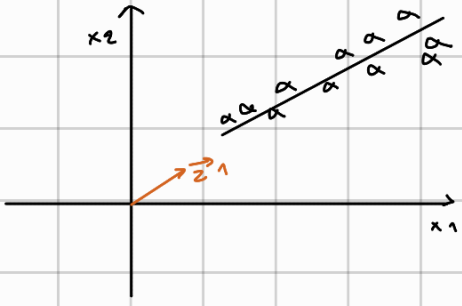
$F_i \leftarrow F \cup x_i$

$\mathcal{L}_i(x|F_i)$

$i = \text{argmin}\ \mathcal{L}_i(x_i|F_i)$

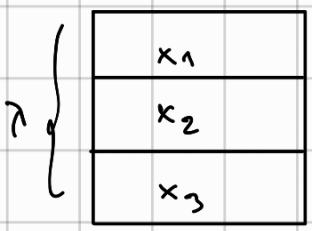
$F \leftarrow F_i$

(3) PCA (Principle component Analyed)



Feature Embedding

$$WDW^T = X \cdot X^T$$



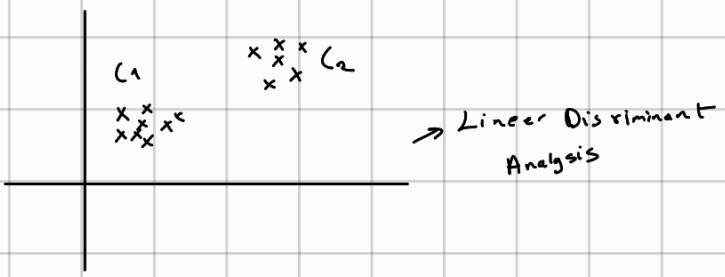
Singular Value Decomposition

SVD: $x = [x_1\ x_2\ \dots\ x_N]_{N\times d}$

$= V A W^T$ → $f \times d$

$N \times f$ $\quad f \times f$

LDA (Linear discriminant Aalysis)



$z = w^T x$

$$m_1 = \frac{\sum_t w^T x^t r^t}{\sum_t r^t} = w^T \cdot \vec{m_1}$$

$$m_2 = \frac{\sum_t w^T r^t (1-r^t)}{\sum_t (1-r^t)} = w^T \vec{m_2}$$

$$s_1^2 = \sum_t (w^T x^t - m_1)^2 r^t$$

$$s_2^2 = \sum_t (w^T x^t - m_2)^2 (1-r^t)$$

$$J(w) = \frac{(m_1 - m_2)^2}{s_1^2 + s_2^2} = \frac{(w^T\vec{m_1} - w^T \cdot \vec{m_2})}{s_1^2 + s_2^2}$$

$$= \frac{w^T (\vec{m_1} - \vec{m_2})^T (\vec{m_1} - \vec{m_2}) w}{s_1^2 + s_2^2}$$

$$= \frac{w^T S_B w}{s_1^2 + s_2^2}$$

$s_1^2 = \sum_t (w^T x^t - m_1)^2 r^t$

$= w^T . S_1 . w$ $\quad S_1 = \sum_t w^t (x^t - \vec{m_1})(x^t - \vec{m_1})^T r^t$

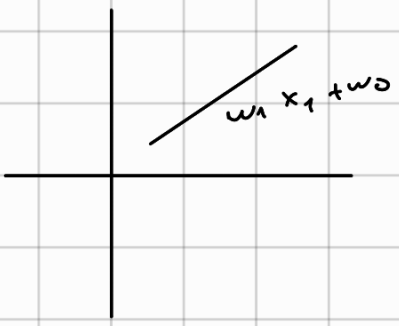
$s_1^2 + s_2^2 = w^T S_w w = \vec{w} (S_1^2 + S_2^2) w$

$J(w) = \frac{w^T S_B . w}{w^T . S_w w}$

$\frac{\partial J(w)}{\partial w} = 0$

$= \frac{w^T (\vec{m_1} - \vec{m_2})}{w^T S_w w}$ 2

Ders 8



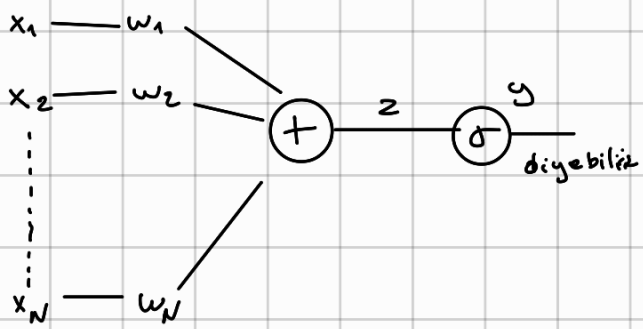
$\vec{w} . \vec{x} + b = \hat{r}^t$

$L(\theta | X) = \sum_{t=1}^{N} (\hat{r}^t - r^t)^2$

türevini alıp 0'a eşitlemek

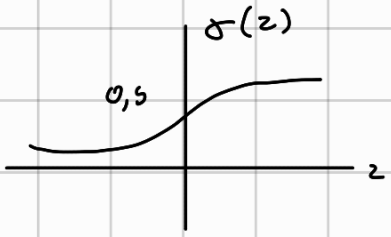
regresyon

Başka gösterim



Sigmoid

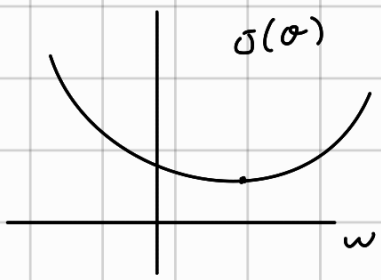
$\sigma(z) = \frac{1}{1+e^{-z}}$



Bu şekilde iteratif olarak ilerliyoruz

gradient desent

$w_i \leftarrow w_i - \eta \frac{\partial J}{\partial w_i}$ (η: lr, J: maliyet fonksiyonu)



Back Propagation

$\hat{y}_i = \sigma(z_i) = \frac{1}{1+e^{-z_i}}$, $z_i = w^T . x_i + b$

$J(w) = -\frac{1}{N} \sum_{i=0}^{N} \underbrace{y_i \log(\hat{y}_i) + (1-y_i) \log(1-\hat{y}_i)}_{\text{cross entropy}}$

$\frac{\partial J(w)}{\partial w} = \frac{\partial J(w)}{\partial \hat{y}_i} \cdot \frac{\partial \hat{y}_i}{\partial z_i} \cdot \frac{\partial z_i}{\partial w}$

önceki — sonra bu — sonra bu

$\frac{\partial J(w)}{\partial \hat{y}_i} = \frac{-y_i}{\hat{y}_i} + \frac{1-y_i}{1-\hat{y}_i}$ $\qquad \frac{\partial \hat{y}_i}{\partial z_i} = \hat{y}_i (1-\hat{y}_i)$

$\frac{\partial z_i}{\partial w} = x_i$

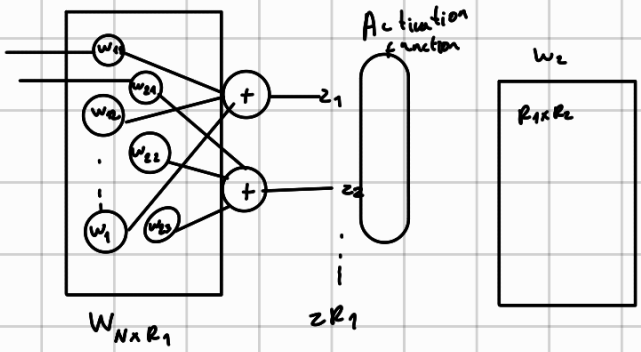
$\frac{\partial J(w)}{\partial w} = \frac{1}{N} \sum_{i=1}^{N} (\hat{y}_i - y_i) \times$ → sonra bunun etkileyeceği z

geri sarmaya çalışıyoruz

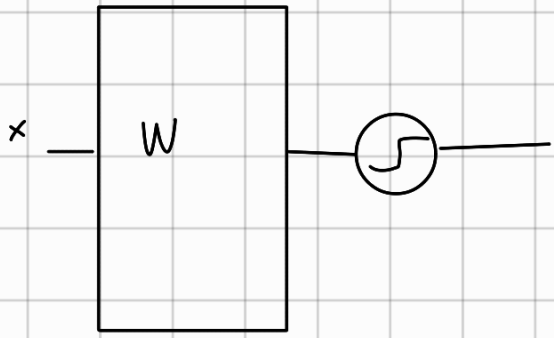
$\frac{\partial J(w)}{\partial b} = \frac{1}{N} \sum_{i=1}^{N} (\hat{y}_i - y_i)$

$x = \{x^t, r^t\}_{t=1}^{M}$ $\qquad P(c_0) = \hat{y}_i$

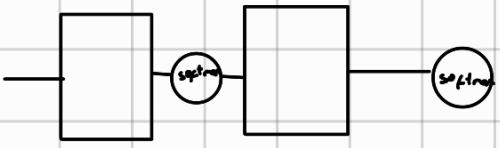
$P(c_1) = (1-\hat{y}_i)$



Activation layers

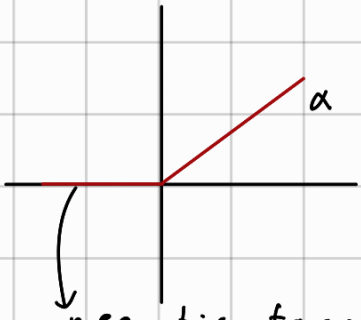


① $softmax = \frac{e^{z_i}}{\sum_{j=1}^{c} e^{z_j}}$ ← olasılık dağılımı



Sonsuza giderken türevi 0 o yüzden iyi çalışmaz. Bu yüzden genelde softmax en son katmanda kullanılır

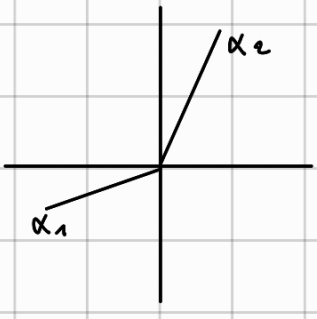
② Relu



axb şeklinde göstermiyoruz non linear

negatif tarafta yine türevi 0, dead neuron sorununa yol açıyor (Vanishing gradiant)

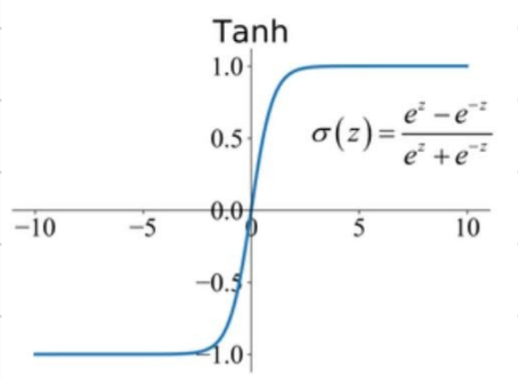
③ Leeky Relu



non linearity

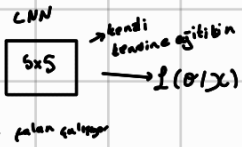
④ $\tanh = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$

burda da türevler $+\infty, -\infty$ da 0'a yaklaşıyor



Ders 9



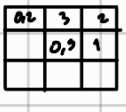


bu W'yi bulan çalışıyor



$\begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix}$ tekbir layer kullanılmaz

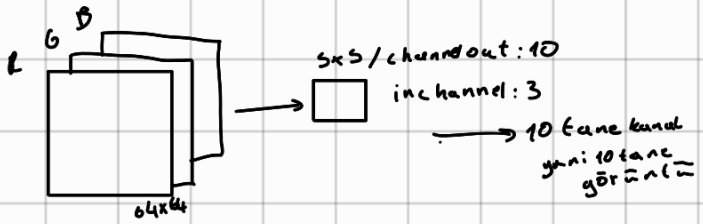
* Pooling layers → Max Pooling
→ Avg Pooling



kernel size filtre sayısı

Feature extraction için çok kullanılır

Odaklanıp işimize yaracağımız verilerin her şey kazandırmıyor.



① feature extraction: Öznitelik çıkarma

② Öznitelikleri konumlarından bağımsız

③ Less parameters

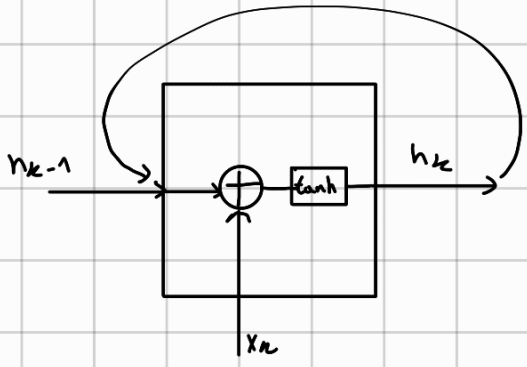
eğitimi anneye göre daha zor

Transfer learning

Elimizdeki veriler ne kadar kaliteli ise o kadar kolay sınıflandırıcı mı zorlar

Yolo feature extraction'ı çok iyi yapıyor ve iyi parçalara ayırıyor

Time series prediction



Patlayan Gradyan (Exploding Gradient):

Geri yayılım sırasında gradyanlar çok büyük değerlere ulaşır ve bu durum ağırlıkların aşırı güncellenmesine neden olur.

Modelin kararsız hale gelmesine yol açar.

Eğitim süreci başarısız olur

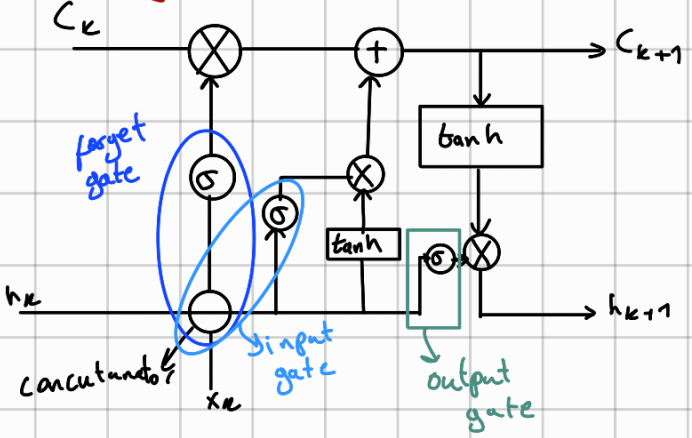
Yok olan Gradyan (Vanishing Gradient):

Geriye yayılım sırasında gradyanlar çok küçük değerlere düşer. Bu durumda ağırlıklar güncellenemez ve model, özellikle uzun dizilerde zaman bağımlılıklarını öğrenemez.

Nedeni ise RNN geri yayılım sırasında zincir kuralını kullanır. Bu süreçte zincirleme türev alınır:

$$\frac{\partial L}{\partial w} = \prod_t \frac{\partial h_t}{\partial h_{t-1}}$$

LSTM



Encoding

ham bir veriyi sayıya çevirmek

kedi: 0

köpek 1

One hot Encoding

R: 0 G: 1 B: 2

R = [1, 0, 0]

G = [0, 1, 0]

B = [0, 0, 1]

Alitopuat

↳ tokenization

Al/i/ /to/ pu/ /a/t/

gözetimsiz olarak entropy kullanddar

king - man = qeen - woman

en basit şekilde nasıl vektöre çevirerim

	D_1	D_2	- - - -	D_N
t_1	0	5		3
t_2				
⋮				
t_N				

TF-IDF

$TF(t,d)$

$IDF(t)$

Term freq (TF)

bazı çok kullanılan kelimeler var

NLP (Natural language processing)

Tokenization

My name is Mehmed ⊙ ?

token

Morfolojik gitmek istersek (dile göre)

Benim adım Mehmet.

Benim adım

kelimelere bölerek word piece

Morfolojik gramere göre

Türkçede 10000 kelime

10 000

ahmet → 2000

Kone hot encoding

$\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$

TF-IDF

Elimizde dokümanlar var

	D_1	D_2	- - -
D_1		3	5

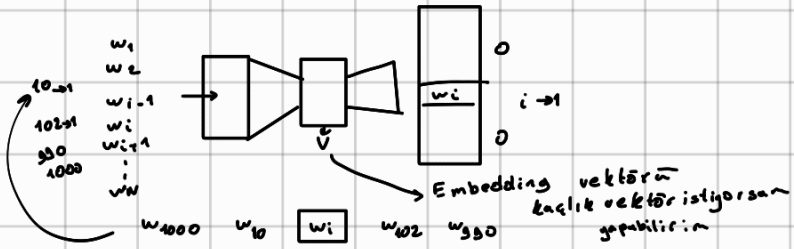
↙ TF Term frequency

Tüm dokümanlarda iki kelime aynı frekansta geçmemeli

Bow TF

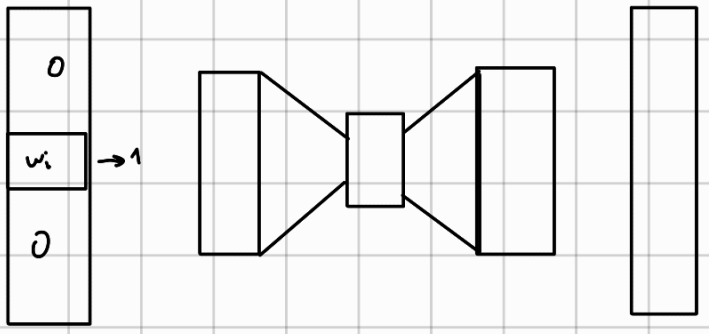
Word2Vec

CBOW Continuos Bag of Words



Skip gram / n-gram 2 gram iki komşuluk

tersten de yapabilirim



Global vectors for word Representation (Glove)

$$J = \sum_{ij} f(x_{ij}) \left(w_i^T \tilde{w}_j + b_i + b_j - \log X_{ij} \right)^2$$

w_i^T: bunu bulmaya çalışıyoruz, kelimenin ağırlığı

$\tilde{w}_j$: i'inci doküman j'inci kelimenin geçme frekansı

$\log X_{ij}$

apple = $t_1 + t_2$

app/le → subword

t_1 t_2

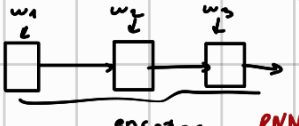
word embedding

Fast Text

Static embedding

Covid geldi yeni kelime

Benim adım Mehmet /Seq 2Seq ↙ Supervized



RNN → exploding vanishing unutma fazla kalıcı

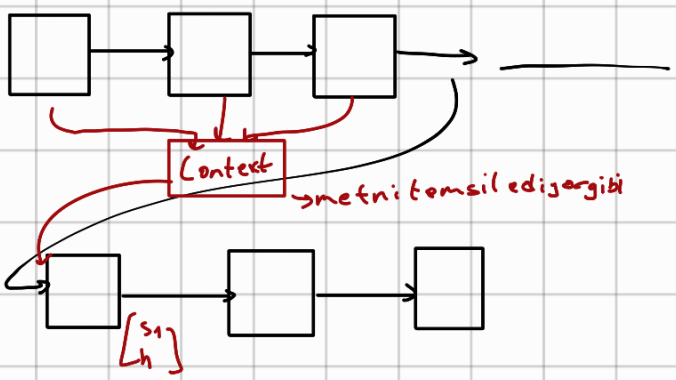
My name is Mehmet

$\tilde{w}_1$ $\tilde{w}_2$ $\tilde{w}_3$ $\tilde{w}_4$

sonraki güncellemede LSTM

Bu metin kısa olmadıktan sonra context yakalayamıyor.

Attention Mechanism



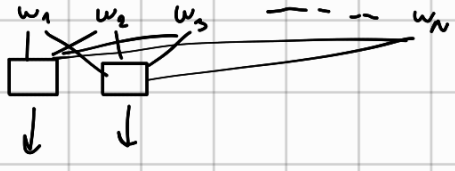
kelimeleri hiden layerle carpiyoruz

$W \cdot h$

softmax → $[\alpha_1, \alpha_2, \ldots, \alpha_{100}]$

hangi kelime ne kadar önemli.

Sonra dönmekgerine MLP koyuyoruz



paralelde calisabiliyor

Encoding :

input encoding

Multi-Head Attention

Feed forward

Dynamic Embedding

Decoder:

output embedding

Multihead Attention

Multihead Attention

Feed Forward

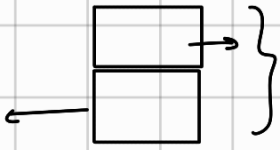
BERT: Sadece Encoder, kelimeleri ayirabiliyor

BERT NLP'nigolosu

BERT : Bidirectional Encoder Representions from Transformers

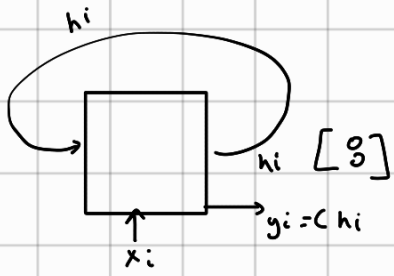
ELMO: Embeddings from Language model

context mekanizmasiyla



RNN





① $[B;A]\begin{bmatrix} x_i \\ hi \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 \\ 2 & 1 & 1 \end{bmatrix}\begin{bmatrix} 3 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 3 \\ 6 \end{bmatrix}$

$C\,hi = [3] = y_i$

② $\begin{bmatrix} 1 & 1 & -1 \\ 2 & 1 & 1 \end{bmatrix}\begin{bmatrix} 4 \\ 3 \\ 6 \end{bmatrix} = \begin{bmatrix} 1 \\ 17 \end{bmatrix}$

$y_2 = 16$

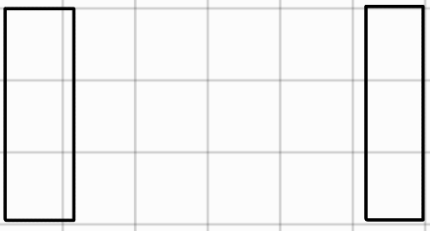
MSE = ?

$\hat{y} = f(CA\begin{bmatrix} x \\ h \end{bmatrix})$

$L = (\hat{y} - y)^2$

I run

I university bus run



Q = Query attention = softmax $\left(\frac{k^T \cdot Q}{\sqrt{d_k}}\right) V$

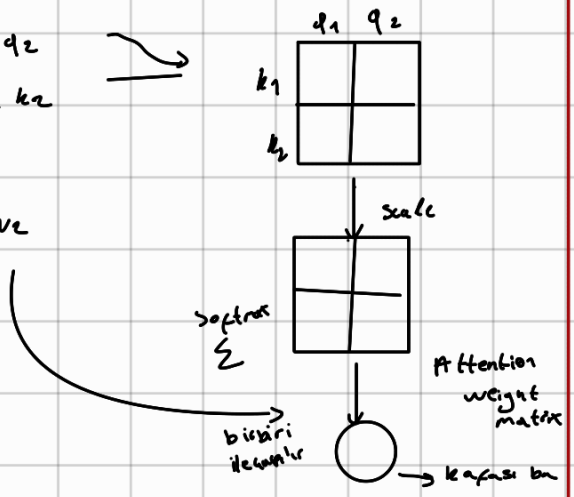
k = key ↓ kökün boyutu

V = Value

$W_Q \cdot w = q_1\ q_2$

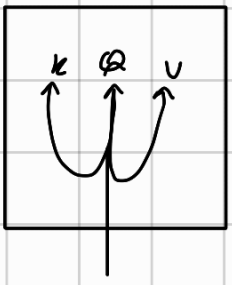
$W_K \cdot w = k_1\ k_2$

$W_V \cdot w = v_1\ v_2$



Bununadı selfattention

Makalelerde böyle görürsünüz

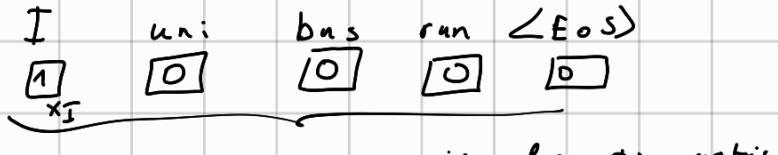


→ multi headde ole olabilir

Q: Nearadığımız

k: Ne ile ilgili

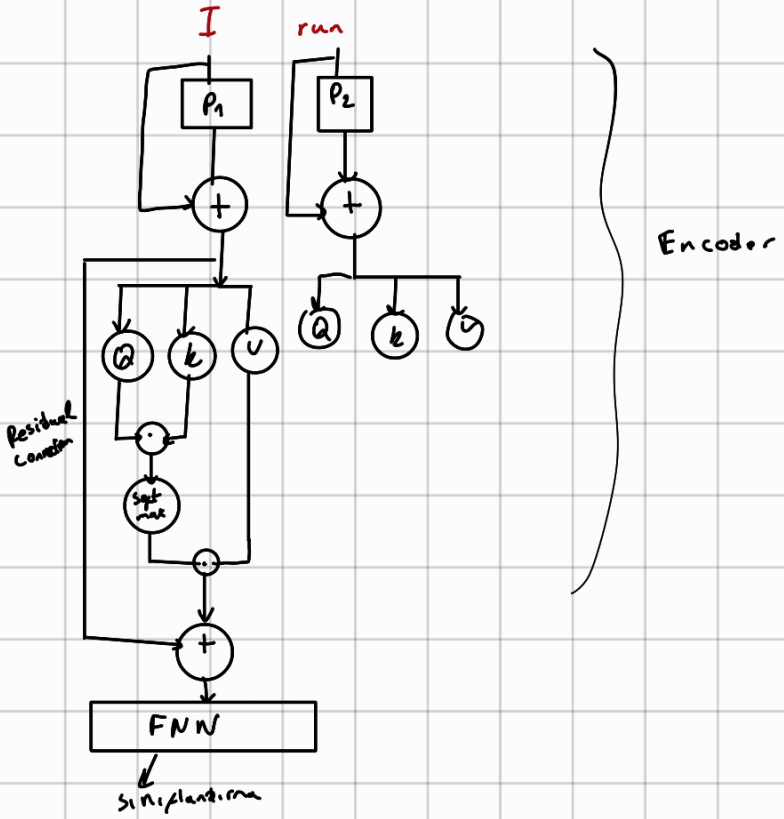
V: Neyi kullanacağız

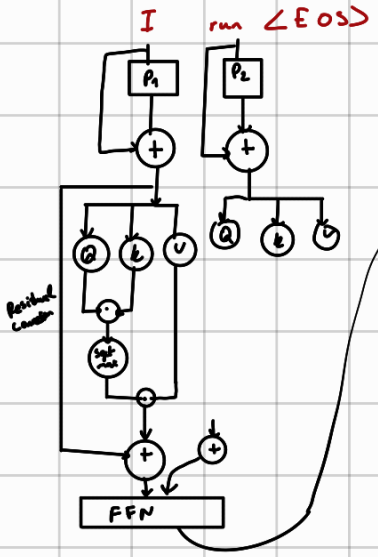


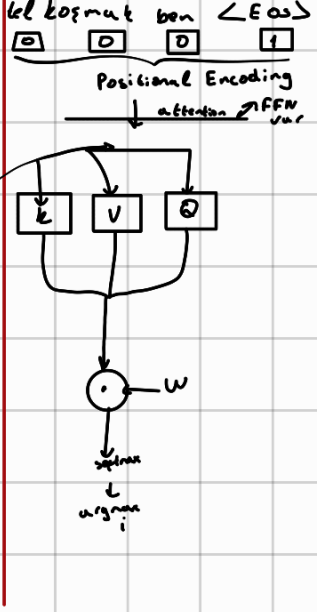
bunun × W_1 → positional encoding matris

$x_I + W_1 x_I = \bar{x_I}$

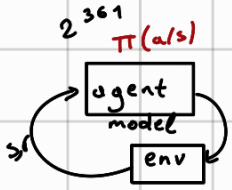
positional embedding







$P(s_{t+1}, a_{t+1} \mid s_t, a_t)$

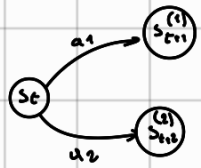


$\pi(a|s)$

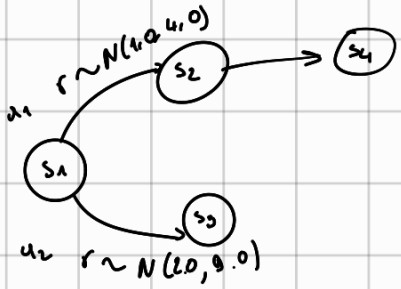
$\pi(s) = a$

$\pi_* \Rightarrow V_*$ → optimum policy

ve aksiyon eylem söyleneli



- Model tabanlı
  Modelsiz



(1)

$Q(s_1, a_1) = 0$

$Q(s_1, a_2) = 0$

SARSA

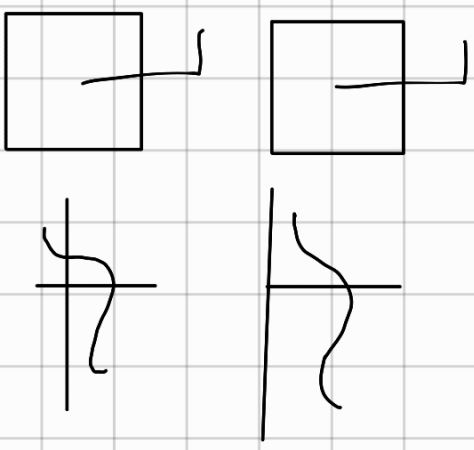
$0 < \alpha < 1$

$Q(s_1, a_1) = Q(s_1, a_1) + \alpha \left(r + V(s_2) - Q(s_1, a_1) \right)$

($Q(s_2, a_2)$; r = reward)

$= 2$

Exploration
Exploitation
Pilemene



Policy gradient theorem

Actor critic (value, policy birleşimi)

$V(s) = E\{G_t \mid S_t\}$

$G_t = \sum_{n=1}^{T} r_{t+n}$

$V(s) = E\{G_t \mid S_t\}$ nasıl hesaplanır

$s_0, a_0, r_1, s_1, a_1, r_2 \ldots$

Gelecekte alacağım ödüllerin ortalaması

Bunun dışında

Multi agent re inforcment learning

$V(s) = E\{G_t \mid S_t\}$

$Q(s, a) = E\{G_t \mid s_t, a\}$

$G_t = \sum_{n=1}^{T} \gamma^{n-1} r_{t+n} = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3}$

$0 < \gamma \leq 1$

$= r_{t+1} + G_{t+1}$

$V(s_t) = r_{t+1} + V(s_{t+1})$